input signal with each of the basis function sinusoids, thus calculating the DFT. Of course, a two-layer neural network is much less powerful than the standard three layer architecture. This means neural networks can carry out *nonlinear* as well as *linear* processing.

Suppose that one of these conventional DSP strategies is used to design the weights of a neural network. Can it be claimed that the network is *optimal*? Traditional DSP algorithms are usually based on assumptions about the characteristics of the input signal. For instance, Wiener filtering is optimal for maximizing the signal-to-noise ratio *assuming* the signal and noise spectra are both known; correlation is optimal for detecting targets *assuming* the noise is white; deconvolution counteracts an undesired convolution *assuming* the deconvolution kernel is the inverse of the original convolution kernel, etc. The problem is, scientist and engineer's seldom have a perfect knowledge of the input signals that will be encountered. While the underlying mathematics may be elegant, the overall performance is limited by how well the data are understood.

For instance, imagine testing a traditional DSP algorithm with actual input signals. Next, repeat the test with the algorithm changed slightly, say, by increasing one of the parameters by one percent. If the second test result is better than the first, the original algorithm is not optimized for the task at hand. Nearly all conventional DSP algorithms can be significantly improved by a trial-and-error evaluation of small changes to the algorithm's parameters and procedures. This is the strategy of the neural network.

Training the Neural Network

Neural network design can best be explained with an example. Figure 26-8 shows the problem we will attack, identifying individual letters in an image of text. This pattern recognition task has received much attention. It is easy enough that many approaches achieve partial success, but difficult enough that there are no perfect solutions. Many successful commercial products have been based on this problem, such as: reading the addresses on letters for postal routing, document entry into word processors, etc.

The first step in developing a neural network is to create a database of examples. For the text recognition problem, this is accomplished by printing the 26 capital letters: A,B,C,D ··· Y,Z, 50 times on a sheet of paper. Next, these 1300 letters are converted into a digital image by using one of the many scanning devices available for personal computers. This large digital image is then divided into small images of 10×10 pixels, each containing a single letter. This information is stored as a 1.3 Megabyte database: 1300 images; 100 pixels per image; 8 bits per pixel. We will use the first 260 images in this database to *train* the neural network (i.e., determine the weights), and the remainder to *test* its performance. The database must also contain a way of identifying the letter contained in each image. For instance, an additional byte could be added to each 10×10 image, containing the letter's ASCII code. In another scheme, the position